# `Terraform-03`

1.Watch the **Terraform-03** video.

| Topic | What to video points |
| --- | --- |
| **Variables** | Reuse Terraform code with different values |
| **Five input methods** | Default, interactive, `-var`, environment variables, `.tfvars` |
| **Variable precedence** | Priority when multiple values are provided |
| **Cross-resource reference** | Access another resource's attributes |
| **Dependencies** | Implicit reference vs explicit `depends_on` |
| **Outputs** | Display selected resource information |
| **State and locking** | Track resources and prevent conflicting changes |

## 2.Execute the script shown in the video.

**Assignment:** Terraform-03
**Environment:** Windows, Visual Studio Code, PowerShell
**Project Directory:** `C:\Users\HP\terraform-03`

---

# Objective

To understand the concepts of Terraform variables, variable precedence, resource attribute references, dependencies, output blocks and Terraform state management.

## Topics Covered

1. Terraform input variables and their purpose.
2. Five different methods of passing Terraform variables.
3. Variable definition precedence.
4. Cross-resource attribute references.
5. Implicit and explicit dependencies.
6. Terraform output variables.
7. Terraform state files and state commands.
8. Remote backend and state locking.

# Objective

To execute Terraform configurations practically using VS Code and verify all five variable methods, variable precedence, dependencies, outputs and state commands.

## Step 1: Project Setup

Created a new Terraform project directory:

`C:\Users\HP\terraform-03`

Created the following configuration files:

- `main.tf` — Terraform resource configuration.
- `variables.tf` — Input variable declarations.
- `dev.auto.tfvars` — Automatically loaded input values, created during Method 5.
- `outputs.tf` — Terraform output definitions, created during the output exercise.

### Commands Executed

terraform init

terraform fmt

terraform validate

terraform plan

terraform apply

**Result:** Terraform initialized successfully, installed the required Local and Random providers, validated the configuration and created the requested resources.

---

# Method 1: Default Values Using variables.tf

## Objective

To pass Terraform variable values using the `default` argument in `variables.tf`.

### variables.tf

```
variable "filename" {
  type    = string
  default = "pets.txt"
}

variable "content" {
  type    = string
  default = "I love cats"
}

variable "prefix" {
  type    = string
  default = "Mr"
}
```

### main.tf

```
resource "local_file" "my_pet" {
  filename = "path.module/{var.filename}"
  content  = var.content
}

resource "random_pet" "petname" {
  prefix    = var.prefix
  separator = "."
```

```
  length    = 1
}
```

### Verification

Get-Content .\pets.txt

terraform state list

### Result

- Created `pets.txt`.
- File content: `I love cats`.
- Generated random pet name: `Mr.iguana`.
- Terraform result: 2 resources added.

**Status: SUCCESS**

---

## Method 2: Interactive CLI Variables

### Objective

To allow Terraform to request values interactively during execution.

### variables.tf

Removed the default values:

```
variable "filename" {
  type = string
}

variable "content" {
  type = string
}

variable "prefix" {
  type = string
}
```

### Command

terraform apply

### Values Provided

| Variable | Input |
|---|---|
| filename | interactive-pets.txt |
| content | Hello from interactive CLI |
| prefix | Miss |

Terraform prompted for the required values because no defaults or other input sources were supplied.

### Verification

Get-Content .\interactive-pets.txt

terraform state list

### Result

- Created `interactive-pets.txt`.
- File content: `Hello from interactive CLI`.
- Resources successfully replaced using interactive input.

**Status: SUCCESS**

---

## Method 3: Command-Line Flags (-var)

### Objective

To pass Terraform variable values directly during command execution without modifying `variables.tf`.

### Command Executed

terraform apply -var="filename=cli-pets.txt" -var="content=Hello from command line" -var="prefix=Dr"

### Verification

Get-Content .\cli-pets.txt

terraform state list

### Result

- Created `cli-pets.txt`.
- File content: `Hello from command line`.
- Terraform result: 2 added, 0 changed, 2 destroyed.

**Status: SUCCESS**

---

## Method 4: Environment Variables (TF_VAR_)

### Objective

To supply Terraform input values through Windows PowerShell environment variables.

### Commands Executed

$env:TF_VAR_filename = "env-pets.txt"

$env:TF_VAR_content = "Hello from environment variables"

$env:TF_VAR_prefix = "Mr"

terraform plan

terraform apply

Terraform automatically picked up the input values from the environment variables.

### Verification

Get-Content .\env-pets.txt

terraform state list

### Result

- Created `env-pets.txt`.

- File content: `Hello from environment variables`.
- Terraform result: 2 added, 0 changed, 2 destroyed.

### Environment Variable Cleanup

Remove-Item Env:TF_VAR_filename

Remove-Item Env:TF_VAR_content

Remove-Item Env:TF_VAR_prefix

**Status: SUCCESS**

---

## Method 5: Automatic Variable Definition File

### Objective

To pass multiple Terraform variables using a separate automatically loaded `.auto.tfvars` file.

### File Created: dev.auto.tfvars

filename = "auto-pets.txt"

content  = "Hello from auto tfvars"

prefix   = "Mrs"

### Commands Executed

terraform fmt

terraform validate

terraform plan

terraform apply

Terraform automatically loaded the values from `dev.auto.tfvars`.

### Verification

Get-Content .\auto-pets.txt

terraform state list

### Result

- Created `auto-pets.txt`.
- File content: `Hello from auto tfvars`.

- Generated pet name: `Mrs.lion`.
- Terraform result: 2 added, 0 changed, 2 destroyed.

**Status: SUCCESS**

---

# Step 2: Variable Definition Precedence

## Objective

To understand which variable value Terraform selects when multiple sources provide values for the same variable.

### Precedence Order (Lowest to Highest)

1. Default values.
2. Environment variables (`TF_VAR_`).
3. `terraform.tfvars`.
4. `terraform.tfvars.json`.
5. `*.auto.tfvars` and `*.auto.tfvars.json`.
6. Command-line `-var` and `-var-file` flags.

## Test 1: Environment Variables vs .auto.tfvars

Configured different environment variable values while the existing `dev.auto.tfvars` file contained its own values.

Executed:

```
terraform plan
```

### Result

```
No changes. Your infrastructure matches the configuration.
```

**Observation:** Terraform selected the values from `dev.auto.tfvars` instead of the environment variables, proving that the automatic variable file has higher precedence.

## Test 2: Command-Line Flags vs .auto.tfvars

Executed:

```
terraform plan -var="filename=cli-priority.txt" -var="content=CLI has highest priority"
-var="prefix=Sir"
```

### Result

Plan: 2 to add, 0 to change, 2 to destroy.

Terraform planned to replace the existing filename, content and random pet prefix with the command-line values.

**Conclusion:** Command-line variable flags override automatically loaded variable files.

Only `terraform plan` was executed for these precedence tests, so no infrastructure changes were applied.

**Status: SUCCESS**

---

# Step 3: Cross-Resource Attribute References

## Objective

To pass information generated by one Terraform resource into another Terraform resource.

### Configuration

```
resource "random_pet" "petname" {
  prefix    = var.prefix
  separator = "."
  length    = 1
}

resource "local_file" "my_pet" {
  filename = "path.module/{var.filename}"
  content  = "${var.content}. My pet name is ${random_pet.petname.id}"
}
```

### Explanation

`random_pet.petname.id` accesses the ID generated by the Random provider.

The `local_file` resource uses that ID to create its file content.

Terraform automatically identifies the relationship between the two resources. This is called an implicit dependency.

### Verification

Get-Content .\auto-pets.txt

### Actual Result

Hello from auto tfvars. My pet name is Mrs.lion

Terraform result:

1 added, 0 changed, 1 destroyed.

**Status: SUCCESS**

---

# Step 4: Explicit Dependency Using depends_on

## Objective

To manually define the dependency between Terraform resources.

### Configuration

Added the following resource to `main.tf`:

```
resource "local_file" "explicit_demo" {
  filename = "${path.module}/explicit-dependency.txt"
  content  = "This file explicitly depends on random_pet"

  depends_on = [random_pet.petname]
```

}

### Explanation

The `depends_on` argument ensures that the `random_pet.petname` resource is created before `local_file.explicit_demo`.

### Commands

terraform plan

terraform apply

Get-Content .\explicit-dependency.txt

terraform state list

### Actual Result

This file explicitly depends on random_pet

Terraform result:

1 added, 0 changed, 0 destroyed.

### Difference Between Dependencies

| Implicit Dependency | Explicit Dependency |
| --- | --- |
| Automatically identified | Manually specified |
| Uses resource attributes | Uses `depends_on` |
| Example: random_pet.petname.id | Example: depends_on = [random_pet.petname] |

**Status: SUCCESS**

---

# Step 5: Terraform Output Variables

## Objective

To display useful resource information after Terraform execution.

### File Created: outputs.tf

```
output "pet_name" {
  description = "Generated random pet name"
  value       = random_pet.petname.id
}

output "pet_file" {
  description = "Path of the generated pet file"
  value       = local_file.my_pet.filename
}
```

### Commands Executed

```
terraform apply
terraform output
terraform output pet_name
terraform output pet_file
terraform output -raw pet_name
```

### Actual Output

```
pet_file = "./auto-pets.txt"
pet_name = "Mrs.lion"
```

For the command:

```
terraform output -raw pet_name
```

The output was:

```
Mrs.lion
```

**Observation:** Output blocks expose selected values from Terraform state without recreating existing resources.

Terraform apply result:

```
0 added, 0 changed, 0 destroyed.
```

**Status: SUCCESS**

# Step 6: Terraform State Verification

## Objective

To inspect managed resources, resource attributes and the current infrastructure state.

### Commands Executed

terraform state list

terraform state show random_pet.petname

terraform show

terraform plan

### Actual State List

local_file.explicit_demo

local_file.my_pet

random_pet.petname

### Random Pet Resource Details

```
resource "random_pet" "petname" {
  id        = "Mrs.lion"
  length    = 1
  prefix    = "Mrs"
  separator = "."
}
```

### Terraform Outputs

pet_file = "./auto-pets.txt"

pet_name = "Mrs.lion"

### Final Plan Result

No changes. Your infrastructure matches the configuration.

**Observation:** Terraform successfully tracked all three resources. The current configuration matched the existing resources, so no changes were required.

**Status: SUCCESS**

---

## Step 7: Remote State and State Locking (Concepts)

## Remote State

Terraform state stores information about resources managed by Terraform. For team environments, state can be stored in remote backends such as Amazon S3, with appropriate access controls and protection.

### State Locking

State locking prevents conflicting concurrent operations against the same Terraform state.

For example, if two engineers execute Terraform operations using the same state, locking helps prevent simultaneous modifications that could corrupt or conflict with the state.

**Note:** Remote state and locking were studied conceptually. No AWS S3 backend or remote locking configuration was created in this practical.

---

# Final Assignment Result

Successfully executed the Terraform-03 practical exercises using Visual Studio Code and PowerShell.

### Completed Activities

- All five Terraform variable methods.
- Variable definition precedence.
- Cross-resource attribute references.
- Implicit resource dependencies.
- Explicit dependencies using `depends_on`.
- Terraform output blocks.
- Terraform state commands and verification.

### Final Verification

| Item | Result |
| --- | --- |
| Terraform resources | 3 |
| Generated pet name | Mrs.lion |
| Main generated file | auto-pets.txt |
| Explicit dependency file | explicit-dependency.txt |
| Output variables | 2 |
| Final Terraform plan | No changes |

# Conclusion

Through this assignment, I gained practical knowledge of passing dynamic values into Terraform, defining resource dependencies, retrieving resource outputs and managing Terraform state.

All executed configurations were successfully verified, and the final Terraform plan confirmed that no further changes were required.

## 3.Integrate Terraform in Jenkins using the Terraform plugin.

### Objective

To integrate Terraform with Jenkins using the Terraform plugin and automatically execute Terraform configuration through a Jenkins Freestyle job.

### 2. Environment Details

| Component | Configuration |
| --- | --- |
| Operating System | Ubuntu Linux (AWS EC2) |
| Jenkins | Existing Jenkins server |
| Jenkins Port | 8080 |
| Jenkins Job | terraform-03-plugin |
| Job Type | Freestyle Project |
| Terraform Version | v1.17.0 |
| Terraform Plugin | v1.0.10 |
| Terraform Executable | `/usr/bin/terraform` |
| Terraform Installation Name | Terraform-CLI |

---

## Step 1: Verify Terraform Installation

Connected to the existing Jenkins Ubuntu EC2 instance.

Executed:

```
which terraform
terraform version
sudo -u jenkins /usr/bin/terraform version
```

**Output:**

```
/usr/bin/terraform
```

Terraform v1.17.0

on linux_amd64

**Result:** Terraform was installed successfully and was accessible to the Jenkins user.

---

## Step 2: Verify Terraform Plugin in Jenkins

1. Opened the Jenkins Dashboard.
2. Navigated to **Manage Jenkins → Plugins**.
3. Selected **Installed plugins**.
4. Searched for `Terraform`.
5. Verified that the Terraform plugin was installed.

**Plugin Details:**

Plugin Name: Terraform Plugin

Version: 1.0.10

No additional installation was required because the plugin was already available from the previous Terraform assignment.

---

## Step 3: Verify Terraform Global Tool Configuration

Navigated to:

**Manage Jenkins → Tools → Terraform installations**

Verified the Terraform installation configuration.

Name: Terraform-CLI

Terraform Home: /usr/bin

Install automatically: Disabled

The existing Terraform executable was reused.

---

## Step 4: Create a Jenkins Freestyle Job

1. Opened Jenkins Dashboard.
2. Clicked **New Item**.
3. Entered the job name `terraform-03-plugin`.
4. Selected **Freestyle project**.
5. Clicked **OK**.

---

## Step 5: Configure Terraform Plugin

Opened the job configuration.

Navigated to **Environment** → **Terraform**.

Configured the following:

| Configuration | Value |
|---|---|
| Terraform | Enabled |
| Terraform Installation | Terraform-CLI |
| Update modules | Unchecked |
| Do not apply automatically | Checked |
| Configuration Type | Configuration Text |
| Resource Variables | Empty |
| Destroy on completion | Disabled |

### Terraform Inline Configuration

Selected **Configuration Text** and entered:

```
resource "terraform_data" "jenkins_test" {
  input = "Hello from Terraform-03 Jenkins Plugin"
}

output "message" {
  value = terraform_data.jenkins_test.output
}
```

### Explanation

- `terraform_data` is a built-in Terraform resource used for this test without creating AWS resources.
- `input` stores the provided message.
- The `output` block displays the message after execution.

---

## Step 6: Configure Jenkins Build Step

Navigated to:

**Build Steps → Add build step → Execute shell**

Added the following shell script:

```
set -eu

echo "===== Terraform-03 Jenkins Plugin Lab ====="

cd "WORKSPACE/terraform - plugin"echo" ===== TerraformVersion ===== ""
TF_BIN" version

echo "===== Terraform Init ====="
"
TF_BIN"init - input = false - no - colorecho" ===== TerraformValidate ===== ""
TF_BIN" validate -no-color
```

```
echo "===== Terraform Plan ====="
"
$TF_BIN" plan -input=false -no-color -out=tfplan
echo "===== Terraform Apply ====="
"$TF_BIN" apply -input=false -no-color tfplan

echo "===== Terraform Output ====="
"$TF_BIN" output -no-color

echo "Terraform-03 Task 3 Completed Successfully"
```

## Script Explanation

| Command | Purpose |
| --- | --- |
| `set -eu` | Stops execution on an error or undefined shell variable |
| `cd "$WORKSPACE/terraform-plugin"` | Opens the plugin's Terraform configuration directory |
| `"$TF_BIN" version` | Checks the Terraform version |
| `terraform init` | Initializes the Terraform working directory |
| `terraform validate` | Validates the configuration |
| `terraform plan -out=tfplan` | Generates and saves an execution plan |
| `terraform apply tfplan` | Applies the saved plan without interactive approval |
| `terraform output` | Displays configured output values |

The Terraform plugin prepares the configuration and provides the Terraform executable reference. The Execute Shell step runs initialization, validation, planning and application.

---

## Step 7: Execute Jenkins Job

1. Clicked **Save**.
2. Opened `terraform-03-plugin`.
3. Clicked **Build Now**.
4. Opened the build's **Console Output**.

### Initial Error

The first execution failed with:

```
Error: error asking for approval: EOF


FATAL: java.lang.Exception:
Terraform Apply failed: 1


Finished: FAILURE
```

### Root Cause

The Terraform plugin was automatically executing:

```
terraform apply -input=false
```

Terraform requested interactive approval, but Jenkins could not provide a response.

### Troubleshooting

Verified the Jenkins job configuration using:

```
sudo grep -n -E -A 5 -B 5 \
'doNotApply|hudson.tasks.Shell|<builders>' \
/var/lib/jenkins/jobs/terraform-03-plugin/config.xml
```

Found:

```
<doNotApply>false</doNotApply>
```

This indicated that automatic Terraform apply was still enabled.

### Solution

1. Opened Jenkins job configuration.
2. Navigated to **Environment → Terraform**.
3. Enabled **Do not apply automatically**.
4. Saved the configuration.

The corrected configuration was:

```
<doNotApply>true</doNotApply>
```

This prevented the plugin from attempting an interactive apply.

The Execute Shell step then applied a saved Terraform plan successfully.

---

## Step 8: Successful Jenkins Execution

Executed the Jenkins job again after correcting the configuration.

### Terraform Initialization

Terraform has been successfully initialized!

### Terraform Validation

Success! The configuration is valid.

### Terraform Plan

Plan: 1 to add, 0 to change, 0 to destroy.

Saved the plan to: tfplan

### Terraform Apply

terraform_data.jenkins_test: Creating...

terraform_data.jenkins_test:

Creation complete after 0s

Apply complete!

Resources: 1 added, 0 changed, 0 destroyed.

### Terraform Output

message = "Hello from Terraform-03 Jenkins Plugin"

### Final Jenkins Result

Terraform-03 Task 3 Completed Successfully

Finished: SUCCESS

---

## Step 9: Final Verification

| Verification | Status |
|---|---|
| Terraform CLI installed | SUCCESS |
| Terraform accessible to Jenkins | SUCCESS |
| Terraform plugin installed | SUCCESS |
| Freestyle job created | SUCCESS |
| Terraform configuration prepared | SUCCESS |
| Terraform initialization | SUCCESS |
| Terraform validation | SUCCESS |
| Terraform plan | SUCCESS |
| Terraform apply | SUCCESS |

| | |
|---|---|
| Output displayed | SUCCESS |
| Jenkins build | SUCCESS |

---

## Conclusion

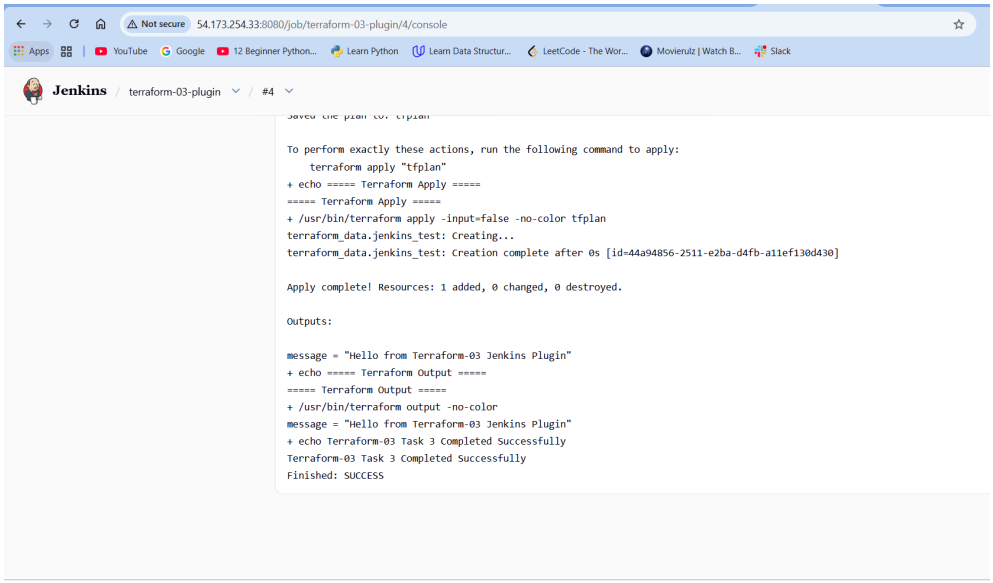
Successfully integrated Terraform with Jenkins using the Terraform plugin v1.0.10.

Created a Jenkins Freestyle job and executed Terraform initialization, validation, planning and application through an Execute Shell build step.

Troubleshot the interactive approval error by disabling the plugin's automatic apply functionality and applying a saved Terraform plan.

The final Jenkins execution completed successfully with one Terraform resource created and the expected output displayed.

**Final Status: TASK 3 COMPLETED SUCCESSFULLY.**




**4.Create one Jenkins job using Maven Project for the code below with two stages:**

- **Stage 1: Git clone**

- **Stage 2: Maven Compilation Code: https://github.com/betawins/java-Working-app.git**

### Objective

To create a Jenkins Maven Project job that performs two operations:

- **Stage 1:** Git Clone — Retrieve Java application source code from GitHub.
- **Stage 2:** Maven Compilation — Execute the Maven compilation goal.

**GitHub Repository:** https://github.com/betawins/java-Working-app.git

---

# Environment Details

| Component | Configuration |
|---|---|
| Operating System | Ubuntu Linux (AWS EC2) |
| Jenkins Version | 2.580.1 |
| Jenkins Port | 8080 |
| Java Version | OpenJDK 21 |
| Maven Version | Apache Maven 3.8.7 |
| Git Version | 2.43.0 |
| Maven Home | `/usr/share/maven` |

| | |
|---|---|
| Jenkins Job Name | terraform-03-maven |
| Job Type | Maven Project |
| Git Branch | main |

---

# Step 1: Verify Java, Maven and Git

Connected to the Jenkins Ubuntu EC2 server.

Executed:

```
java -version

mvn -version

git --version
```

Initially, Maven was not installed.

### Maven Installation

```
sudo apt update

sudo apt install maven -y
```

### Verification

```
mvn -version

which mvn
```

Verified Maven access for the Jenkins user:

```
cd /tmp

sudo -u jenkins mvn -version
```

### Result

Apache Maven 3.8.7

Maven home: /usr/share/maven

Java version: 21.0.12.1

Git version: 2.43.0

**Status: SUCCESS**

---

# Step 2: Configure Maven in Jenkins

1. Open Jenkins Dashboard.
2. Navigate to **Manage Jenkins** → **Tools**.
3. Locate **Maven installations**.
4. Configure the installed Maven executable.

## Configuration

| Field | Value |
|---|---|
| Name | Maven-3 |
| MAVEN_HOME | /usr/share/maven |
| Install automatically | Unchecked |

Clicked **Save**.

---

# Step 3: Create Maven Project Job

1. Open Jenkins Dashboard.

2. Click **New Item**.
3. Enter the job name `terraform-03-maven`.
4. Select **Maven project**.
5. Click **OK**.

**Result:** Jenkins Maven Project job created.

---

# Step 4: Configure Stage 1 — Git Clone

Navigated to:

**Job Configuration → Source Code Management**

Selected **Git**.

### Git Configuration

| Field | Value |
|---|---|
| Source Code Management | Git |
| Repository URL | https://github.com/betawins/java-Working-app.git |
| Credentials | None |
| Branch Specifier | `*/main` |

### Explanation

Jenkins uses Git Source Code Management to retrieve repository code into its workspace before executing Maven.

**Workspace:**

/var/lib/jenkins/workspace/terraform-03-maven

### Verification

Check the beginning of Jenkins Console Output for Git fetch and checkout messages, including the checked-out revision.

**Note:** Git checkout was configured in the job. The screenshot provided shows the end of the build, so the exact checkout revision has not yet been recorded in this documentation.

---

## Step 5: Configure Stage 2 — Maven Compilation

Navigated to:

**Job Configuration → Build**

Configured the following:

| Field | |
|---|---|
| Root POM | |
| Goals and options | |

Maven
Installation

### Maven Command

mvn clean compile

### Explanation

**mvn clean**

Removes the previous build output, typically the `target` directory.

**mvn compile**

Executes the Maven compile phase and compiles applicable Java source files when present.

**pom.xml**

The Project Object Model file contains Maven project configuration, dependencies, plugins and build settings.

**Important:** This is a Jenkins Maven Project job, not a Pipeline job. Git checkout and Maven compilation are configured as two sequential operations rather than explicitly named Pipeline stages.

---

## Step 6: Execute the Jenkins Job

1. Clicked **Save**.

2. Opened the `terraform-03-maven` job.
3. Clicked **Build Now**.
4. Opened Build **#1**.
5. Selected **Console Output**.

### Actual Maven Console Output

[INFO] No sources to compile

[INFO] BUILD SUCCESS

[INFO] Total time: 13.658 s

Jenkins also displayed Maven POM archiving activity.

### Final Jenkins Result

Finished: SUCCESS

---

## Step 7: Verify the Build

| Verification | Result |
|---|---|
| Maven Project job created | SUCCESS |
| Git repository configured | SUCCESS |
| Branch configured | main |
| Root POM configured | pom.xml |

| | |
|---|---|
| Maven compile goal executed | SUCCESS |
| Java source files compiled | No sources to compile |
| Jenkins Build #1 | SUCCESS |

### Observation: No Sources to Compile

The Maven Console Output displayed:

> [INFO] No sources to compile

This means Maven successfully executed its compile phase but did not find Java source files to compile in its configured source directories.

This message is not a build failure.

The Maven build completed successfully, although actual Java source compilation was not demonstrated.

---

## Step 8: Final Result

**Job Name:** terraform-03-maven

**Repository:** https://github.com/betawins/java-Working-app.git

**Maven Goals:** clean compile

**Build Number:** #1

**Build Status:** SUCCESS

**Total Maven Build Time:** 13.658 seconds

---

## Conclusion

Successfully created and configured a Jenkins Maven Project using the provided GitHub repository.

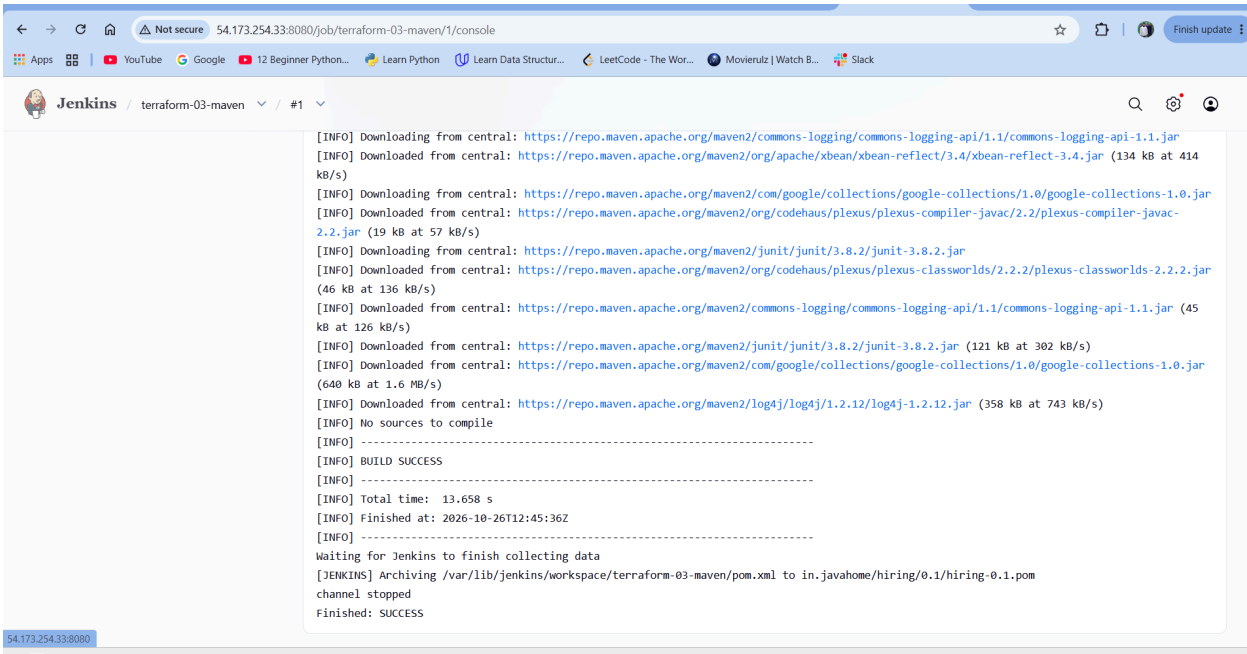
Configured Git Source Code Management for repository checkout and Maven goals for the compilation phase.

Executed the Jenkins job and verified that Maven completed successfully.

The build returned `BUILD SUCCESS` and Jenkins displayed `Finished: SUCCESS`.

**Final Status: TASK 4 COMPLETED AT THE JENKINS BUILD LEVEL.**

*Git checkout can be independently confirmed from the beginning of the Console Output. The compile phase reported no Java sources to compile.*




## 5.Use the same code and create a parameterized job in Jenkins with:

- **Stage 1: Git clone**
- **Stage 2: Maven Compilation Code: https://github.com/betawins/java-Working-app.git**

---

# Environment Details

| Component | Configuration |
| --- | --- |
| Operating System | Ubuntu Linux (AWS EC2) |
| Jenkins Version | 2.580.1 |
| Jenkins Port | 8080 |
| Java Version | OpenJDK 21 |
| Maven Version | Apache Maven 3.8.7 |
| Git Version | 2.43.0 |
| Job Name | terraform-03-parameterized |
| Job Type | Declarative Pipeline |
| Parameter | BRANCH_TO_BUILD |
| Default Branch | main |

| | |
|---|---|
| Maven Command | mvn -B clean compile |

---

# Step 1: Create a Jenkins Pipeline Job

1. Open Jenkins Dashboard.
2. Click **New Item**.
3. Enter the job name `terraform-03-parameterized`.
4. Select **Pipeline**.
5. Click **OK**.

**Result:** Created a new Jenkins Pipeline job.

---

# Step 2: Configure Build Parameters

Navigated to:

**General → This project is parameterized**

Enabled the checkbox.

Clicked **Add Parameter → String Parameter**.

## Parameter Configuration

| Field | Value |
|---|---|
| Name | BRANCH_TO_BUILD |
| Default Value | main |
| Description | Enter the Git branch to build |

| | |
|---|---|
| Trim the string | Enabled, if available |

### Explanation

Parameters allow Jenkins users to provide different input values during job execution.

For this assignment, the `BRANCH_TO_BUILD` parameter controls which Git branch Jenkins checks out.

Example:

BRANCH_TO_BUILD = main

---

## Step 3: Configure Jenkins Pipeline

Navigated to:

**Pipeline → Definition → Pipeline script**

Enabled **Use Groovy Sandbox**.

Added the following Declarative Jenkins Pipeline.

### Jenkins Pipeline Script

```
pipeline {
    agent any

    stages {

        stage('Git Clone') {
            steps {
                echo "Cloning branch: $params.BRANCH_TO_BUILD" git branch: "
{params.BRANCH_TO_BUILD}",
```

```
                url: 'https://github.com/betawins/java-Working-app.git'

            sh 'git rev-parse --short HEAD'
            sh 'test -f pom.xml'

            echo 'Git Clone Successful'
        }
    }

    stage('Maven Compilation') {
        steps {
            echo 'Starting Maven Compilation'

            sh 'mvn -B clean compile'

            echo 'Maven Compilation Completed'
        }
    }
}

post {
    success {
        echo 'Parameterized Jenkins Build SUCCESS'
    }

    failure {
```

```
            echo 'Parameterized Jenkins Build FAILED'
        }
    }
}
```

---

## Step 4: Stage 1 — Git Clone

### Objective

To clone the selected branch from GitHub into the Jenkins workspace.

### Pipeline Code

```
stage('Git Clone') {
    steps {
        echo "Cloning branch: $params.BRANCH_TO_BUILD"$gitbranch: "
{params.BRANCH_TO_BUILD}",
            url: 'https://github.com/betawins/java-Working-app.git'

        sh 'git rev-parse --short HEAD'
        sh 'test -f pom.xml'

        echo 'Git Clone Successful'
    }
}
```

### Explanation

- `params.BRANCH_TO_BUILD` reads the branch name supplied by the user.
- The `git` step retrieves the selected branch from GitHub.
- `git rev-parse --short HEAD` displays the short Git commit hash.
- `test -f pom.xml` checks that the Maven POM file exists.

- The success message is displayed when all previous steps complete successfully.

**Jenkins Workspace:**

/var/lib/jenkins/workspace/terraform-03-parameterized

**Result:** The Pipeline proceeded to Maven compilation and completed successfully, indicating the checkout stage did not stop the build.

The exact Git checkout revision should be verified from the beginning of the Jenkins Console Output.

---

# Step 5: Stage 2 — Maven Compilation

## Objective

To execute Maven compilation after successfully retrieving the GitHub code.

## Pipeline Code

```
stage('Maven Compilation') {
    steps {
        echo 'Starting Maven Compilation'

        sh 'mvn -B clean compile'

        echo 'Maven Compilation Completed'
    }
}
```

## Maven Command

```
mvn -B clean compile
```

## Explanation

**Purpose**

Executes Apache Maven

Enables batch mode, suitable for CI/CD automation

Removes previous Maven build output

Runs the Maven compilation phase

## Actual Console Output

[INFO] --- maven-compiler-plugin:3.1:compile (default-compile) @ hiring ---

[INFO] No sources to compile

```
[INFO] BUILD SUCCESS


[INFO] Total time: 0.775 s
```

### Observation

Maven executed its compilation phase successfully.

The message `No sources to compile` indicates that Maven did not find Java source files in the configured compilation source directories.

The Maven build was successful, although no Java source files were compiled during this execution.

---

## Step 6: Configure Post-Build Actions

The Pipeline includes a `post` block.

```
post {
    success {
        echo 'Parameterized Jenkins Build SUCCESS'
    }


    failure {
        echo 'Parameterized Jenkins Build FAILED'
    }
}
```

### Explanation

- `success` runs when the Pipeline completes successfully.
- `failure` runs when the Pipeline fails.
- These messages help identify the final build status.

---

## Step 7: Execute the Parameterized Job

1. Click **Save**.
2. Open the `terraform-03-parameterized` Jenkins job.
3. Click **Build with Parameters**.
4. Enter the required branch.

### Parameter Value

BRANCH_TO_BUILD = main

5. Click **Build**.
6. Open Build **#1**.
7. Click **Console Output**.

---

## Step 8: Verify Jenkins Build Result

### Actual Console Output

[INFO] No sources to compile

[INFO] BUILD SUCCESS

Maven Compilation Completed

Parameterized Jenkins Build SUCCESS

[Pipeline] End of Pipeline

Finished: SUCCESS

### Build Verification

| Verification | Status |
| --- | --- |
| Parameterized Pipeline configured | SUCCESS |
| BRANCH_TO_BUILD parameter | Configured |
| Git Clone stage | Completed without reported failure |
| Maven Compilation stage | SUCCESS |
| Maven Build | BUILD SUCCESS |
| Jenkins Pipeline | Finished: SUCCESS |

---

## Step 9: Difference Between Task 4 and Task 5

| Task 4 | Task 5 |
| --- | --- |
| Maven Project | Parameterized Pipeline |
| Git branch configured directly | Git branch supplied as parameter |

| | |
|---|---|
| Root POM and Maven goals configured in UI | Git and Maven commands written in Jenkinsfile-style Pipeline code |
| Build Now | Build with Parameters |
| Fixed branch configuration | User-selectable branch |

### Advantages of Parameterized Jenkins Jobs

1. The same Jenkins job can be reused with different input values.
2. Users can choose Git branches without modifying Pipeline code.
3. Reduces the need to create separate jobs.
4. Improves flexibility and reusability.
5. Useful for DevOps CI/CD automation.

---

## Step 10: Final Result

**Jenkins Job:** terraform-03-parameterized

**Job Type:** Parameterized Declarative Pipeline

**GitHub Repository:** https://github.com/betawins/java-Working-app.git

**Branch Parameter:** BRANCH_TO_BUILD

**Parameter Value:** main

**Build Number:** #1

**Maven Build Result:** BUILD SUCCESS

**Jenkins Final Status:** SUCCESS

---

## Conclusion

Successfully created a parameterized Jenkins Pipeline job using the given GitHub repository.

Configured a string parameter named `BRANCH_TO_BUILD` to select the Git branch dynamically.

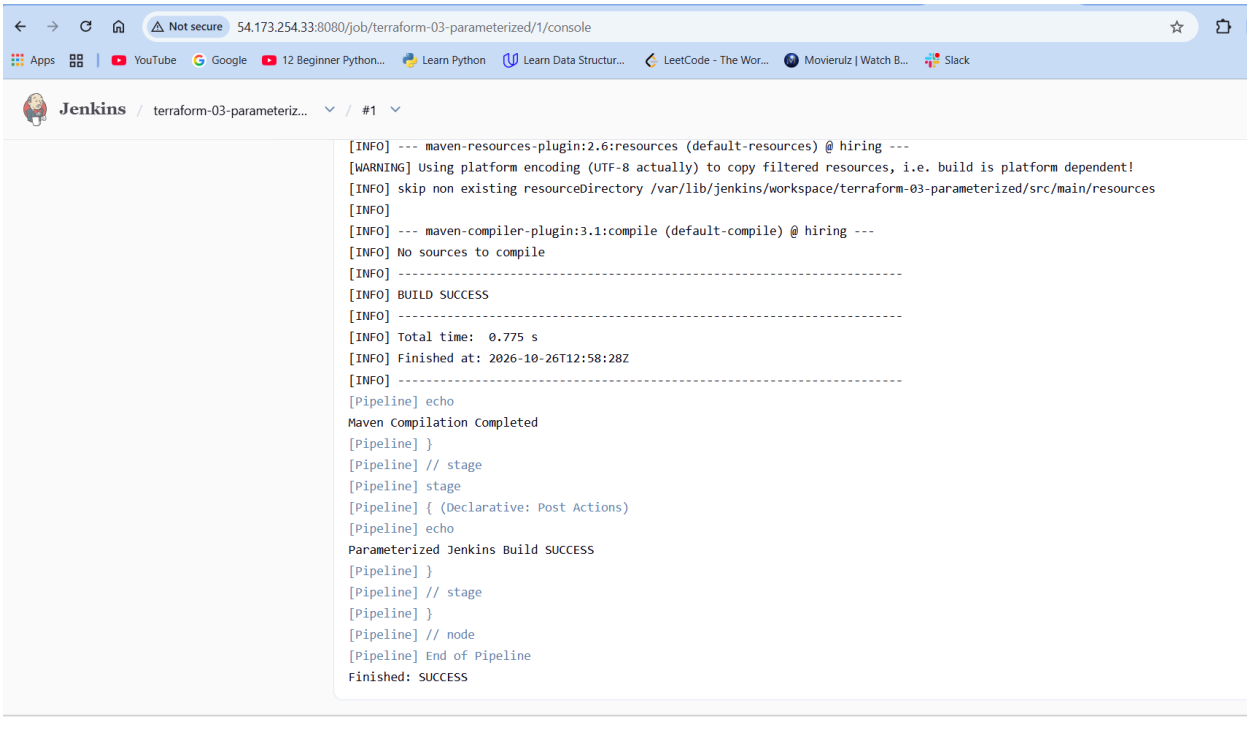
Implemented two Pipeline stages:

1. Git Clone
2. Maven Compilation

Executed the Jenkins Pipeline using the `main` branch parameter.

The Maven compilation goal completed successfully, and Jenkins reported `Finished: SUCCESS`.

**Final Status: TASK 5 COMPLETED SUCCESSFULLY.**




## What are the global variables in Jenkins?

# What are Global Variables in Jenkins?

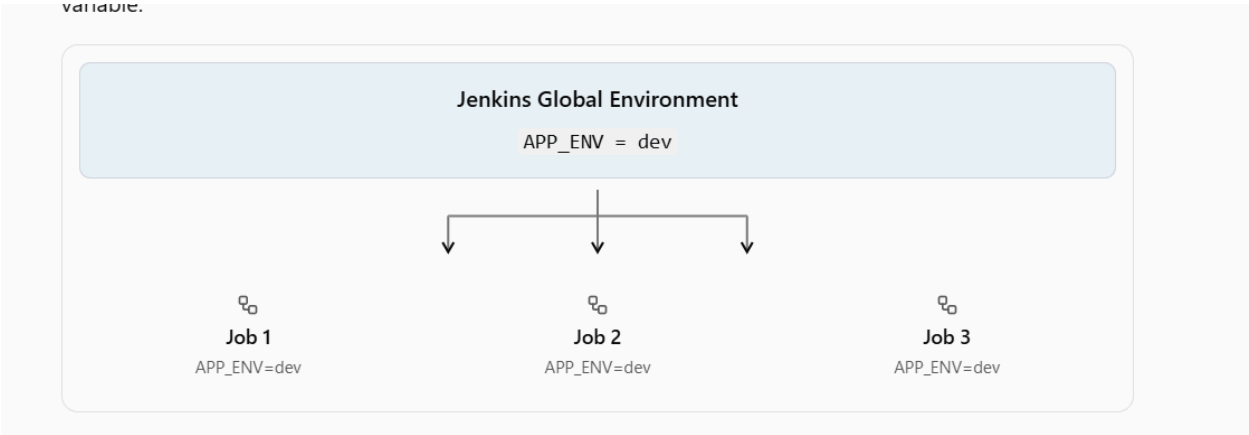
Global variables in Jenkins are variables that can be accessed across multiple Jenkins jobs or pipelines, depending on their scope and configuration.

In simple words, instead of defining the same value separately in every Jenkins job, we can define it centrally and reuse it.

### Real-time example

Suppose your company has 10 Jenkins jobs, and all jobs use the same application environment.

Instead of defining `APP_ENV=dev` separately in all 10 jobs, you can configure it as a global environment variable.




## 2. Important built-in Jenkins variables

Jenkins automatically provides several environment variables during builds. These are available through the Pipeline's `env` object.



Jenkins
+1

| Variable | Purpose | Example |
| --- | --- | --- |
| `JOB_NAME` | Current Jenkins job name | terraform-03-parameterized |

| | | |
|---|---|---|
| `BUILD_NUMBER` | Current build number | 1 |
| `BUILD_ID` | Current build identifier | 1 |
| `BUILD_URL` | Link to the build | Jenkins build URL |
| `WORKSPACE` | Job workspace directory | `/var/lib/jenkins/workspace/...` |
| `NODE_NAME` | Node executing the job | built-in |
| `JENKINS_URL` | Jenkins server URL, if configured | Jenkins URL |
| `BUILD_TAG` | Unique build tag | jenkins-job-1 |
| `EXECUTOR_NUMBER` | Executor running the build | 0 |

## 3. How to create a custom global variable

On your existing Jenkins server:

1. Open Manage Jenkins → System.
2. Find Global properties.
3. Enable Environment variables.
4. Click Add.
5. Enter the following:

| Name | Value |
| --- | --- |
| APP_ENV | dev |
| PROJECT_NAME | Java-Working-App |

6. Click Save.

These variables will be available to jobs that inherit Jenkins' global environment configuration.


Jenkins

# 4. Practical Jenkins Pipeline example

You can use this sample to understand how global variables work.

```
pipeline {
    agent any

    stages {
        stage('Display Jenkins Variables') {
            steps {
                echo "Job Name: ${env.JOB_NAME}"
                echo "Build Number: ${env.BUILD_NUMBER}"
                echo "Workspace: ${env.WORKSPACE}"
                echo "Node: ${env.NODE_NAME}"

                echo "Environment: ${env.APP_ENV}"
                echo "Project: ${env.PROJECT_NAME}"
            }
        }
    }
}
```

If you configure the custom variables above, this Pipeline will print them alongside Jenkins' built-in variables.

In Linux Execute Shell steps, you can access these using `$JOB_NAME`, `$BUILD_NUMBER` and similar names.

## 5. Global variable vs parameterized variable

You already worked with a parameter named `BRANCH_TO_BUILD` in Task 5.

| Global environment variable | Parameterized variable |
| --- | --- |
| Configured centrally | Configured for a job |
| Reusable across multiple jobs | Supplied for an individual build |
| Example: `APP_ENV=dev` | Example: `BRANCH_TO_BUILD=main` |
| Access using `env.APP_ENV` | Access using `params.BRANCH_TO_BUILD` |

Important: Don't store passwords or GitHub tokens as ordinary global variables. Use Jenkins Credentials instead.

## 6. What to write for Task 6 documentation

Objective: Understand Jenkins global variables and their use in CI/CD automation.

Explanation: Jenkins provides built-in environment variables such as `JOB_NAME`, `BUILD_NUMBER`, `WORKSPACE`, `BUILD_URL`, and `NODE_NAME`. These help identify and manage build execution. Administrators can also configure custom global environment variables under Manage Jenkins → System → Global properties.

Example: `env.BUILD_NUMBER` retrieves the current Jenkins build number, while `env.WORKSPACE` identifies the directory where the build is running.

Advantages: Global variables improve configuration consistency, reduce repeated hardcoded values and make Jenkins jobs easier to maintain.

Conclusion: Global variables are useful for sharing common configuration values and accessing Jenkins build information across CI/CD workflows.

Q: What are global variables in Jenkins?

Global variables are commonly used environment values that Jenkins jobs and pipelines can access. Jenkins provides built-in variables such as `JOB_NAME`, `BUILD_NUMBER` and `WORKSPACE`. We can also define custom global environment variables through Jenkins system configuration. These help reuse values and simplify automation.